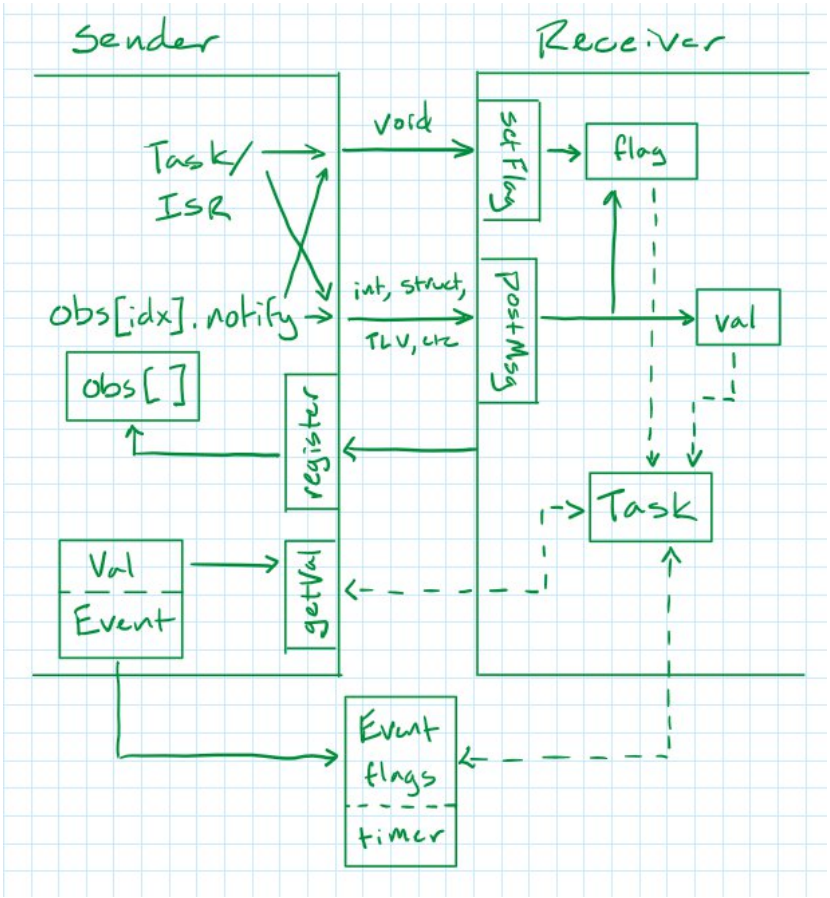
Message Passing with Pointers

Nathan Jones

When designing a complex embedded system it is considered a Very Good Thing™ to both (1) separate the system into distinct tasks and (2) require that those tasks communicate, not through global variables, but through a queue or function (i.e. “messages”). Together, those design guidelines help prevent “spaghetti code” (and a host of other architectural flaws) by forming the embedded system from a set of isolated modules that interact with each other using distinct interfaces. Modern RTOSs provide several different mechanisms to facilitate this, such as thread flags, event flags, semaphores, and message queues. The graphic below shows (generically) what it might look like for two tasks to communicate using these mechanisms (from “You Don’t Need an RTOS (Part 3)”[1]).



“Sending” tasks can communicate Boolean values (an event happened, something is/is not true) to “receiving” tasks by setting their thread flags (*setFlag()*) or by setting a global event flag (which the receiver would be monitoring). They can also communicate larger values (integers, structs, etc) by calling a function (*postMsg(val)*), which might then operate on the data right

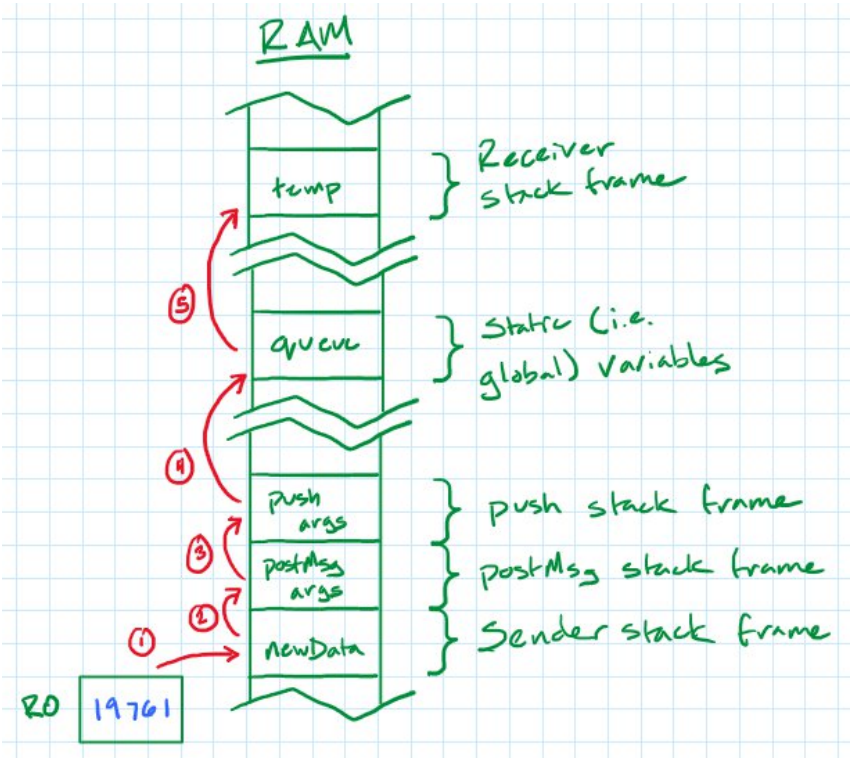
[1] Jones, Nathan. 2024. “You Don’t Need an RTOS (Part 3).” Embeddedrelated.com. https://www.embeddedrelated.com/showarticle/1653.php.

away, copy the function argument to a local variable, or add it to a thread-local queue or "fire-place buffer"[2] to be processed at a later time.

As messages get large, however, passing them all by value to each receiving task could incur a significant memory and performance cost, by virtue of those larger messages being copied to different sections of memory over and over again. Consider the following "worst case" scenario in which a task generates random values, storing them in a local struct, before sending them to another task by calling a function called *postMsg()*[3]; *postMsg()* wraps a function call to the actual queue function, *push()*. The receiving task temporarily stores the arrays in a queue (using *push()*), eventually pulling them out of the queue (*pop()*) and copying them to its own local variable before processing them[4].

In this configuration, each randomly-generated value is copied to a different section of memory ***five separate times***:

1. to the sender's local array (when the value is first generated),
2. to the stack (when calling the receiver's *postMsg()* function),
3. to the stack again (when *postMsg()* calls *push()* to add the value to the queue),
4. to the receiver's queue (inside *push()*), and finally
5. to the receiver's local variable (when the data is "popped").



If, instead, the sender gave a pointer to the data[5] to *postMsg()*, this would eliminate ***all but one*** of the five memory copies, replacing them with copying just the pointer to the data instead. (This, of course, happens implicitly in C if an array is given as a function argument.) On Com-

---

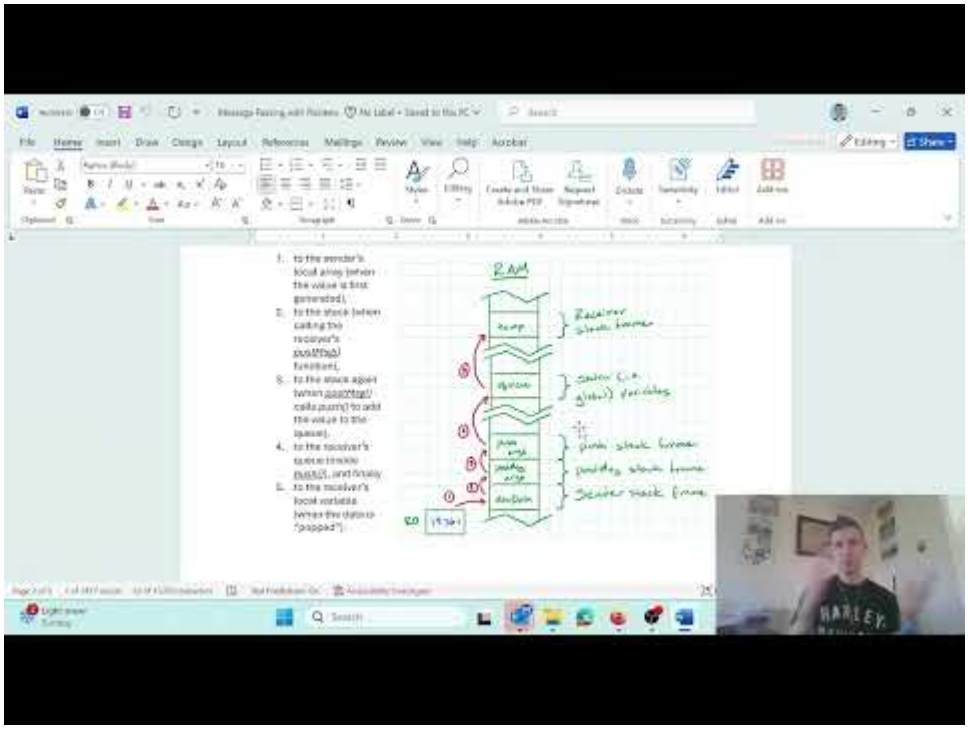
[2] Sachs, Jason. 2020. "Scorchers, Part 3: Bare-Metal Concurrency with Double-Buffering and the Revolving Fireplace." July 5, 2020. https://www.embeddedrelated.com/showarticle/1360.php.

[3] Is the call to *postMsg* strictly required? No, not really, but it's definitely good design practice. Leaving it out would save one memory copy, at the cost of tightly coupling the sending task to the specific queue API being used in the application.

[4] https://godbolt.org/z/jcccfTrd6; note that in this example the "queue" is just a local variable, which is like a queue with space for a single element.

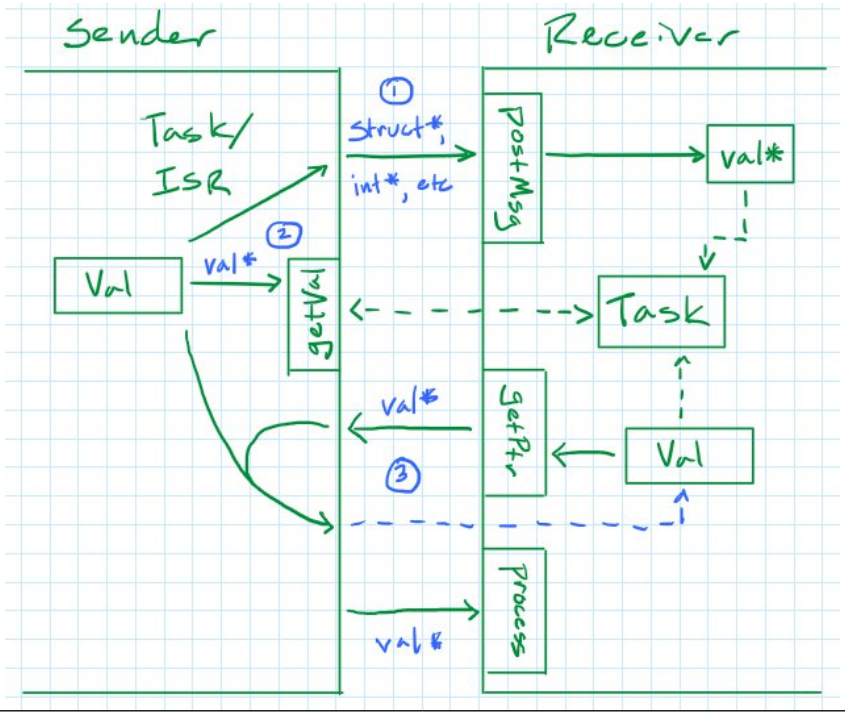
[5] https://godbolt.org/z/4j9sjzMTr; note that in this example the "queue" is just a local variable, which is like a queue with space for a single element.

piler Explorer (see footnotes 3 and 4), passing around a pointer instead of the struct resulted in 25% fewer LOC. And that's just for a struct that has a 16-element array of integers! The amount of memory space and the number of instruction cycles you save increases if the data you're passing around is larger than that. Passing around pointers to data also has the benefit of making it easier to treat data polymorphically; it's slightly less complicated to create a generic pointer to different data types than it is to create a single data type that can be used polymorphically.



https://www.youtube.com/watch?v=ipc7SJVx_r4

There are three different ways we can pass around pointers to data (depicted below).



(1) Sender allocates memory, copies data, then sends pointer to receiver

```
void sender(void)
{
  while(1)
  {
    bigStruct_t* new = malloc(...);
    if(new)
    {
      *new = /* Fill with data */;
      while(!postMsg(new));
    }
  }
}
```

```
bigStruct_t* curr_data = NULL;

bool postMsg(bigStruct_t* data)
{
  bool success = false;
  if(curr_data == NULL)
  {
    curr_data = data;
    success = true;
  }
  return success;
}

void receiver(void)
{
  while(1)
  {
    while(curr_data == NULL);
    /* Process data */;
    free(curr_data);
    curr_data = NULL;
  }
}
```

(2) Sender allocates memory and copies data. Receiver requests data and receives a pointer.

```
bigStruct_t new;
bool freed = true;

void sender(void)
{
  while(1)
```

```
void receiver(void)
{
  while(1)
  {
    bigStruct_t* data = getVal();
    if(data)
```

```
  {
    while(!freed);
    new = /* Fill with data */;
    freed = false;
  }
}

bigStruct_t* getVal(void)
{
  return !freed ? &new : NULL;
}

void free(bigStruct_t* data)
{
  if(data == &new)
  {
    memset(&new, 0, sizeof(bigStruct_t));
    freed = true;
  }
}
```

```
    {
      /* Process data */
      free(data);
    }
  }
}
```

(3) Receiver allocates memory. Sender requests a pointer, copies data, then notifies receiver.

```
void sender(void)
{
  while(1)
  {
    bigStruct_t* new = getPtr();
    if(new)
    {
      *new = /* Fill with data */;
      process(new);
    }
  }
}
```

```
bigStruct_t new = {0};
bool freed = true;

bigStruct_t* getPtr(void)
{
  bigStruct_t* ret = NULL;
  if(freed)
  {
    ret = &new;
    freed = false;
  }
  return ret;
}

void process(bigStruct_t* data)
{
  /* Process data */;
  memset(&new, 0, sizeof(bigStruct_t));
  freed = true;
}
```

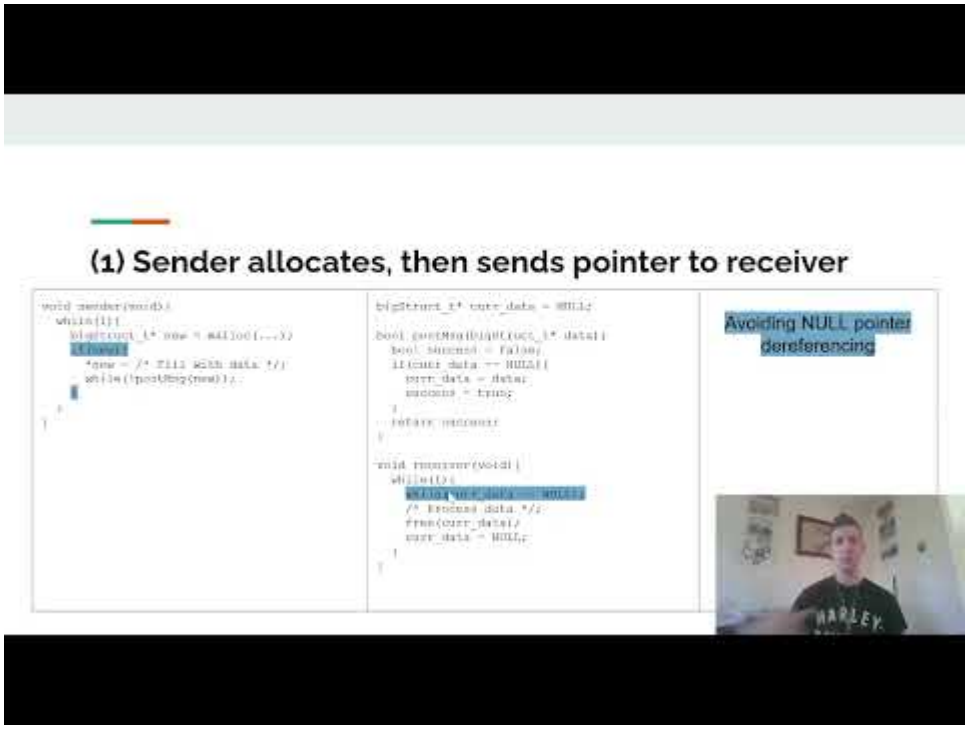
The first two solutions above operate just like sending or requesting an actual value (*void postMsg(bigStruct_t)* or *bigStruct_t getVal(void)*), except a pointer is used (*void postMsg(bigStruct_t*)* or *bigStruct_t* getVal(void)*). In both cases, the sender allocates the memory for the data (either statically or dynamically). The third solution is different in that the *receiver* allocates memory for the data to be received. The sender requests a pointer to this piece of memory (or the receiver pushes a pointer to the sender) and then copies its data there. Once it is finished, the sender returns the pointer to the receiver to let it know that it can process the data (e.g. *void process(bigStruct_t*)*). Although I've only shown each pointer being copied to a local variable in the examples above, the receiver could just as easily add the pointers to a queue for processing at a later time, freeing them once its finished with its processing.

The cost of saving memory and instruction cycles by passing around pointers to memory, though, is the added complexity of correctly using those pointers. Incorrect use of pointers could result in such errors as the ones listed below.

- NULL pointer dereferencing

- memory leaks
- race conditions
- dangling pointers

To avoid **NULL pointer dereferencing**, tasks that allocate memory need to handle possible failures in allocation, even if it's from a static memory pool (even the pool could be exhausted at some point). Note how the sender (in the first and third examples above) and the receiver (second example) only use their pointers to data if they have a non-NULL value.

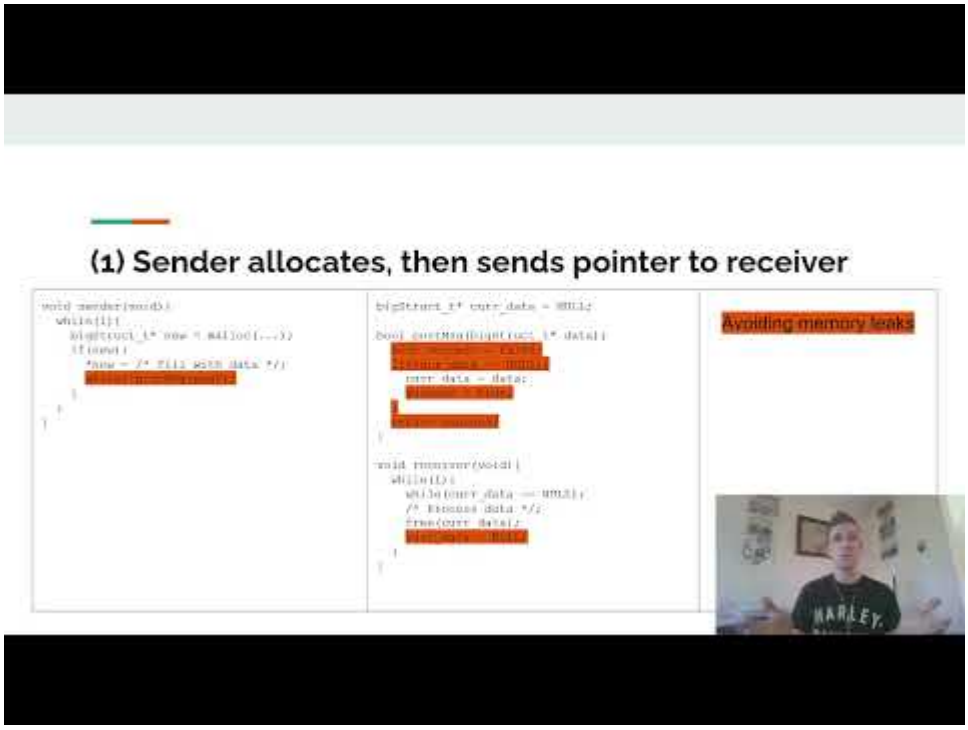


https://www.youtube.com/watch?v=BUDHrUjO56w

To avoid **memory leaks**, pointers that are allocated dynamically (even if that's from a static memory pool) *must* be freed after the system is finished using them. Thus, the receiving tasks need to know how to free or recycle the memory once they're done using it. If the sender allocates the memory for the data (as in the first two examples, above), the simplest solution is for the sender to use dynamic memory (*malloc()*/*calloc()*), which the receiver would then *free()* once it was done using the data (as in the first example). If the sending task has allocated its memory differently (from, say, a static pool of objects), then it should expose a "free" function that the receiving task could use for the same purpose (as in the second example). Memory recycling gets triggered implicitly with the third solution, by virtue of the sender returning a pointer to the receiver. After the receiver processes the data, it should recycle its own memory in whatever way it needs (in this example, by zeroing out memory and resetting *freed* to *true*).

Additionally, the first example is susceptible to *another* type of memory leak, which would occur if the sending task tried to give a pointer to a receiver that had no room to store said pointer. This could occur in the first example if a sender were to call *postMsg()* before the receiver had finished processing an earlier pointer (i.e. when *curr_data != NULL*). In that case, *postMsg()* should *probably* return an error value and, if it does, the sending task needs to handle that error

rather than ignoring it. The sending task can do that by either (1) repeatedly attempting to re-send the current pointer (waiting for a spot to open up) or (2) by freeing the pointer it's trying to send (i.e. abandoning the current operation) and starting again at the top of its task loop[6].



https://www.youtube.com/watch?v=DZaTYqsvhRM

To avoid **race conditions**, the sender must abide by the rule that once it's given a pointer to the receiving task, it can no longer access that data (until the receiver frees it). If you're using C++11 or later, you might consider using *std::unique_ptr* to enforce this. If the pointer to your data is a "unique pointer" then it's not possible to *copy* the reference anywhere, only to *move* it. Once moved, the previous owner of the unique pointer is given the value *nullptr*, preventing any further access to the data in question.

Of course, avoiding **race conditions** also involves analyzing your code to identify all places where preemption by an ISR or higher-priority task could cause problems. In each of the examples above, there's at least one spot where two lines of code are intended to be atomic and where havoc could be raised if they were interrupted! In the first example, *postMsg()* checks if *curr_data* is *NULL* and, if it is, copies the function argument to it. A problem could occur, however, if *immediately* after *curr_data == NULL* is evaluated as TRUE, *another* (higher-priority) sending task called *postMsg()*. If that happens, then the higher-priority task's pointer would get overwritten by the first sending task's pointer (once the higher-priority task completed and the first sender was allowed to continue its execution), resulting in a memory leak and a lost piece

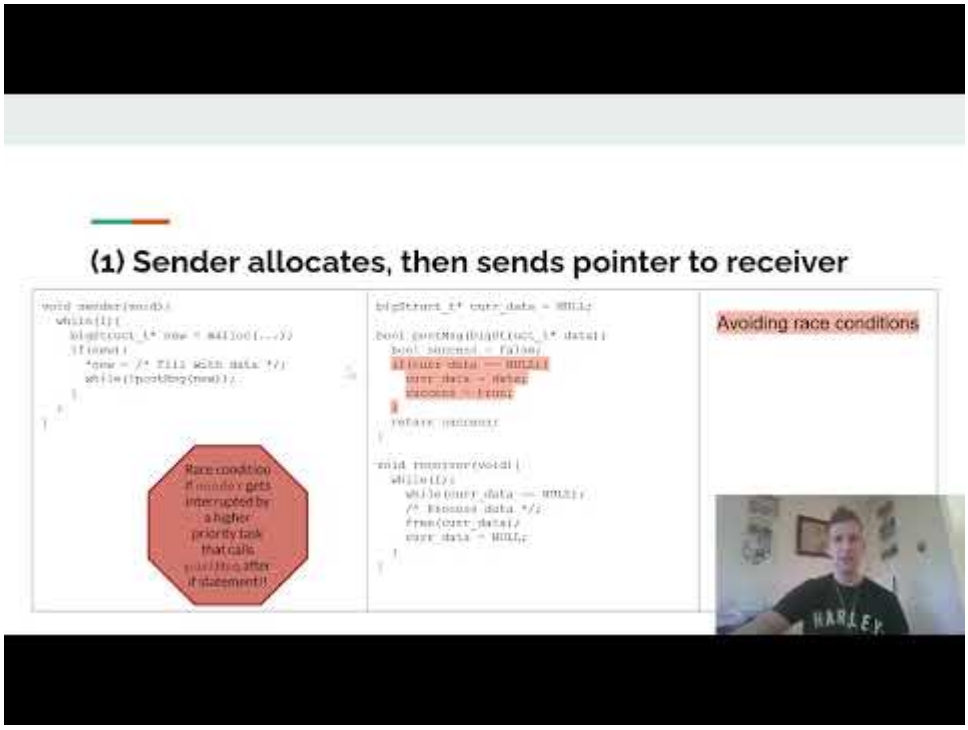
---

[6] It's also possible, though a touch more complicated, to configure *postMsg()* to simply overwrite old values with new ones (freeing them correctly, of course!). The added complexity comes from the fact that we wouldn't want *postMsg()* or *process()* overwriting a pointer that the receiving task was currently using, so we'd need another piece of synchronization to prevent that.

of data. A similar problem occurs in the the third example (inside *getPtr()*). Solving this problem involves forcing the offending code to be executed atomically, either by:

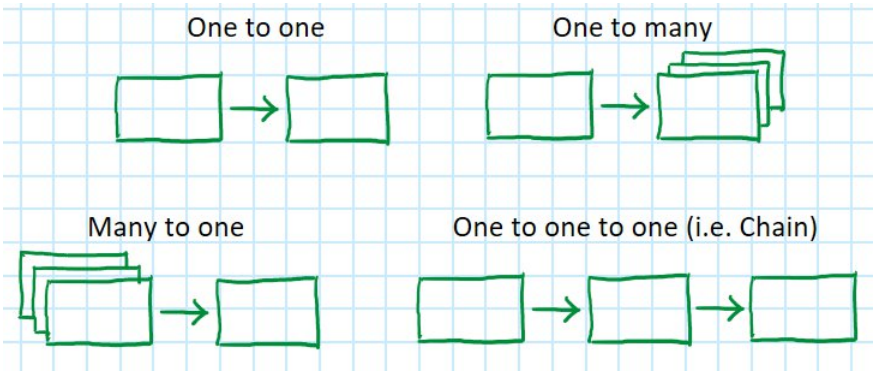
- disabling interrupts before the critical section of code (and re-enabling them after),
- acquiring a mutex before executing the critical section, or
- using atomic operations such as *atomic_compare_exchance_strong* from *stdatomic.h*.

There's a lot more that could be said about identifying and fixing race conditions, but that would deserve its own article!



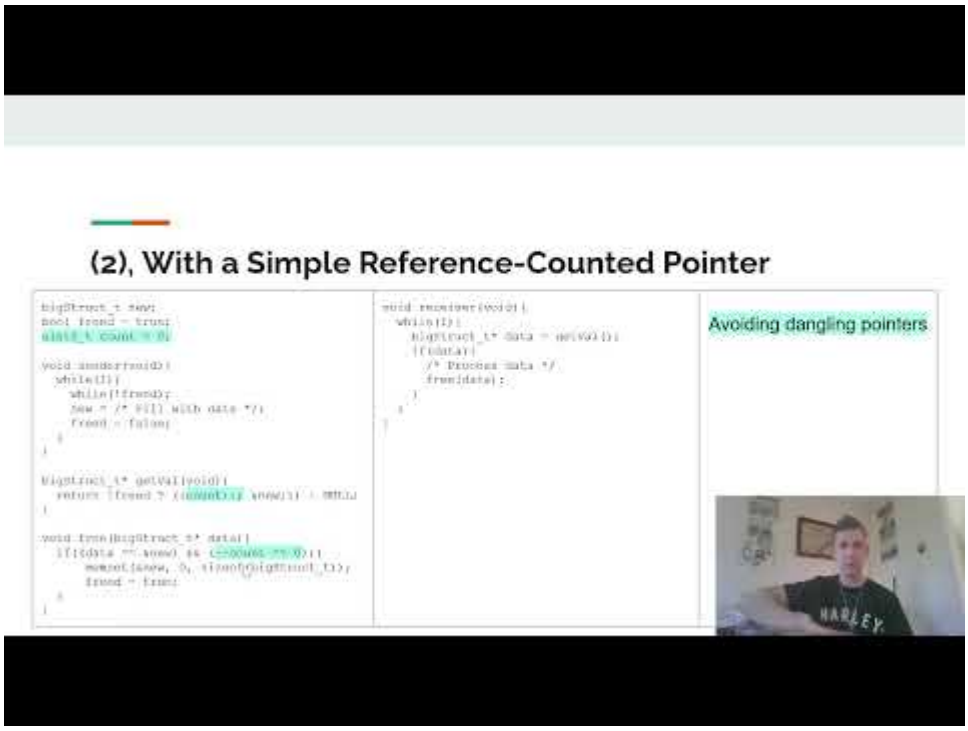
https://www.youtube.com/watch?v=v5PwjCAIb90

There are a few additional considerations to be made when there are multiple receivers ("one to many"), multiple senders ("many to one"), or if the data is being processed in a chain of tasks ("one to one to one").



If there are **multiple receivers**, each with their own pointer to the same piece of data, then a simple *free()* function (that recycles the memory unconditionally when it's called) would leave

dangling pointers after it was called by the first task! Instead, the application should use something called a “reference counted” pointer, which is a type of pointer that keeps track of how many references to it have been given out. The first task to free the pointer simply decrements the counter value; it’s only when the *last* task frees the pointer that the memory is actually recycled. If you’re using C++11 or later, you can accomplish this with *std::shared_ptr*. If not, you’ll just need to roll your own[7].



https://www.youtube.com/watch?v=nhmgpErxbZU

Furthermore, its most likely in this situation that each of the receivers is only *supposed* to have read-only access to the underlying data. You can’t actually prevent a receiver from modifying the data once it has a pointer to it[8], but defining the pointer as a “pointer to a *constant* DATA” will help avoid well-meaning programmers from *accidentally* modifying it. You can do this by adding *const* in front of your pointer declaration. For example, *const int* * is a “pointer to a constant int” (as is *int const* *); note that this is different from *int* * *const*, which is a “constant pointer to a (non-constant) int”.

The third solution above is *less* suited for this configuration with multiple receivers, since the sending task would still need to copy its data into each sender’s pointer. However, it *is* well-suited for the configuration with **multiple senders** (“many to one”) since the sender knows how to recycle its own data. The problem with using either of the first or second solutions in that configuration is that the receiving task needs to know how to free the pointers when it’s done with the data. If the heap (*malloc*/*free*) is not being used, then each sending task would need to also

---

[7] Mailund, Thomas. 2020. “Reference counting lists in C.” Mailund.dk. 2020. https://mailund.dk/posts/c-refcount-list/.
[8] Unless your microcontroller has an MMU and you put your data inside a read-only block of memory

give the receiving task pointers to their individual “free” functions, which would complicate things.

If your tasks are organized like a **chain**, with each task performing their operation on the data until the end of the chain is reached, then any of the three solutions above will work fine, though I would recommend that only the very first or the very last task allocate the memory (or, possibly, another module that just handles memory allocations), rather than each task in the entire chain.

To summarize, message passing helps prevent “spaghetti code” and passing pointers to data (instead of the data itself) helps limit the amount of memory and the number of instruction cycles needed for the tasks in your application to communicate with each other. Sending around pointers to data, though, is more complicated than just passing around the data by value, and the system designer needs to ensure that when they do so, they avoid NULL pointer dereferencing, memory leaks, race conditions, and dangling pointers. Additionally, some of the three solutions proposed above require modification if (or are simply not well-suited when) there are multiple receivers, multiple senders, or if tasks are arranged in a chain.

If you’ve made it this far, thanks for reading and happy hacking!